

Here's a **line-by-line explanation** of your C++ program that implements **sequential and parallel versions of Bubble Sort and Merge Sort**, with a menu-driven interface:

Header Files

```
cpp
CopyEdit
#include <iostream>           // for input/output operations
#include <vector>             // to use vector container
#include <thread>             // for multithreading
#include <future>             // for asynchronous execution (std::async)
#include <chrono>             // for timing the sort execution
```

Sequential Bubble Sort

```
cpp
CopyEdit
void bubbleSort(vector<int>& arr) {
    ...
}
```

- Basic Bubble Sort: Compares adjacent elements and swaps if necessary.
 - $n - i - 1$ reduces comparisons as largest values "bubble up".
-

Parallel Bubble Sort (Odd-Even Transposition Sort)

```
cpp
CopyEdit
void parallelBubbleSort(vector<int>& arr) {
    ...
}
```

- Loops n times (like regular Bubble Sort).
 - Alternates between:
 - Even indexed comparisons (i.e., `arr[0] vs arr[1]`, `arr[2] vs arr[3]`)
 - Odd indexed comparisons (`arr[1] vs arr[2]`, ...)
 - Each comparison-swap is done in **parallel** using `std::async`.
-

Merge Function (for Merge Sort)

```
cpp
CopyEdit
void merge(vector<int>& arr, int left, int mid, int right) {
    ...
}
```

- Merges two sorted subarrays:
 - `leftArray = arr[left to mid]`
 - `rightArray = arr[mid+1 to right]`
 - Merges them into `arr[left to right]`.
-



Sequential Merge Sort

```
cpp
CopyEdit
void mergeSort(vector<int>& arr, int left, int right) {
    ...
}
```

- Standard divide-and-conquer:
 - Divide the array into halves recursively.
 - Merge the sorted halves.
-



Parallel Merge Sort

```
cpp
CopyEdit
void parallelMergeSort(vector<int>& arr, int left, int right) {
    ...
}
```

- Similar to sequential, but the left and right halves are **sorted in parallel** using `std::async`.
 - Synchronization is done using `future.get()` before merge.
-



Measure Time Function

```
cpp
CopyEdit
long long measureTime(function<void()> sortMethod) {
    ...
}
```

- Measures the execution time of any given function.
 - Uses `chrono` to record time before and after execution.
-



Utility: Print Array

```
cpp
CopyEdit
```

```
void printArray(const vector<int>& arr) {
    ...
}
```

- Prints array elements space-separated.

Main Program

```
cpp
CopyEdit
int main() {
    ...
}
```

Menu System

```
cpp
CopyEdit
while (continueRunning) {
    ...
}
```

- Displays options:
 1. Sequential Bubble Sort
 2. Parallel Bubble Sort
 3. Sequential Merge Sort
 4. Parallel Merge Sort
 5. Exit

For Options 1–4:

```
cpp
CopyEdit
vector<int> arr(size);
cin >> arr[i];
```

- Takes user input and duplicates array for fair comparison.

Sorting Logic

```
cpp
CopyEdit
measureTime([&]() { sortFunc(); });
```

- Calls the respective sort and times it.
- Sorted array and time are displayed.

Exit Logic

```
cpp
CopyEdit
cout << "Do you want to continue? (yes/no): ";
```

- Loop continues based on user input.

☒ Exit

```
cpp
CopyEdit
return 0;
```

- Program ends gracefully.
-

Summary of Sorts:

Sort Type	Technique Used	Parallelism
Bubble Sort (Seq)	Nested loops	✗
Bubble Sort (Par)	Odd-Even transposition	☒ via async
Merge Sort (Seq)	Recursive divide & conquer	✗
Merge Sort (Par)	Parallel recursion + merge	☒ via async